SPIRENT Communications Analyze | Assure | Accelerate™

Software Design
Phoenix Automation Framework
P2-Sal Interface Wrappers

Document Number: xxx-xxxx-xxx
Revision 1.0
April 11, 2006

Approvals:

Prepared by: Kevin Guerra Senior Software Developer	Prepared by:
TBD Name TBD Title	TBD Name TBD Title
TBD Name TBD Title	TBD Name TBD Title

TABLE OF CONTENTS

`

1.0 Introduction

1.1 Scope

The scope of this document is to describe the Sal Interface Wrappers project of the Phoenix Automation Framework (PAF). This document is very brief since it describes a project which its sole purpose is to wrap the SAL (Spirent Automation Layer.) More information about SAL can be obtained in separate documents. This document describes the interaction between user scripts, such as tcl or Perl, and SAL.

1.2 Project Description

Through this project Phoenix version 2 will provide automation capabilities that will meet the goals for SR12. SAL will provide a thin interface to the BLL from scripting languages using SWIG to automatically generate interfaces.

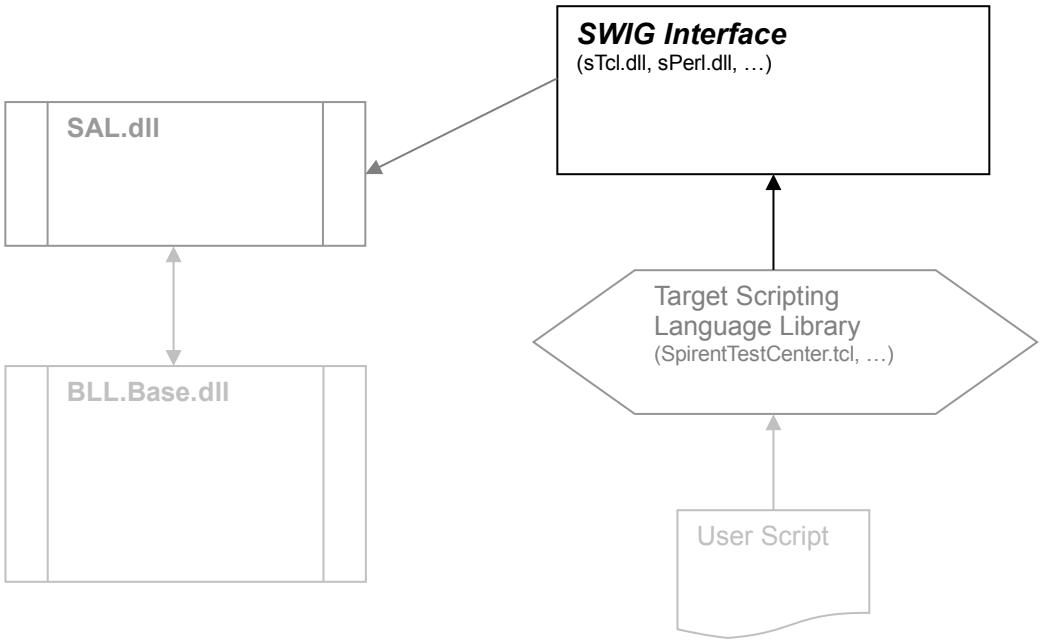
Much of the code in the old tcl libraries will disappear, as it no longer applies to P2, including all code for the SMB namespace and utilities. The remaining code will be migrated to the C++ BLL interface project or SAL.

Our goal is to be able to generate interfaces to diverse scripting languages automatically and very little if any code should be part of those scripting libraries. This will allow us to write code that is shared, will perform more efficiently and will be easier to maintain.

2.0 Component Design

This component consists of one C++ module, a scripting library that makes the module accessible in a hopefully consistent way across different scripting languages implementation, and the actual SAL DLL which provides the actual interface to P2 through a defined API.

The following diagram contains the proposed modules, including existing ones:



2.1 Existing modules: SAL and BLL.Base

Bll.Base is Spirent's Business Logic Layer. For more information please review applicable documents. This module is used by SAL through the messaging service. BLL.Base will respond to requests from SAL and can send messages and event notifications to SAL.

SAL is the new Spirent's Automation Layer, or Framework as it used to be called PAF (Phoenix Automation Framework. The aim of SAL is to contain "all" the logic for any scripting language interface. For more information on SAL please review its documentation.

The only important part to discuss about SAL here is its interface. SAL's interface is defined in SalInterface.h and in it you'll find all the functions that are exportable from the SAL.dll, including it's initialization, shutdown and logging functions.

In SAL's interface code, in the cpp file, the exportable functions deal with setting up the necessary objects during Init() and deletion of them during shutdown. Errors are handled via the standard and Spirent supplied exception handling mechanisms from C++. All or at least most exceptions are caught here, then whatever exception type it may be, it is converted into a C++ standard exception which is then thrown to the caller. In this case the caller would be the SWIG Interface Module.

2.2 SWIG Interface Module

Module actually is somewhat of a misnomer, although it is a module by virtue of “being” a dll. This module at the moment has no code other than one function that is run during dll load, and that is the Init() function.

The goal was to have as little code as possible in this module. Therefore, this module references a header file from SAL, SalInterface.h. There is another header file and one cpp code file for this project, and they are empty except for the inclusion of other header files. In any event here is the code for the whole project, except for SalInterface.h, which can vary as needed.

The example code given here is for sTcl.dll:

//sTcl.h
```
#include "../sal/include/salInterface.h"
```

//sTcl.cpp
```
#include "sTcl.h"
```

//sTcl.i
```
%module sTcl

%{
#include "sTcl.h"
#include "../sal/include/salInterface.h"
%}

%include "std_string.i"
%include "std_vector.i"
%include exception.i

namespace std {
   %template(StringVector) vector<string>;
}

%exception
{
   try
   {
      $action
   }
   SWIG_CATCH_STDEXCEPT
}

%include "sTcl.h"
%include "../sal/include/salInterface.h"

%init
%{
   stc::salInit();
%}
```

When you build this project, this command is run prior to compilation:
```
..\..\..\..\common\tools\swig\swig -c++ -tcl8 -prefix stc -namespace -pkgversion 1.0 $(InputPath)
```

The command produces sTcl_wrap.cxx, which is the file SWIG creates that wraps SalInterface.h and makes available those exportable functions for our scripting language, in this case tcl. From sTcl.i interface file, one should note that we request SWIG to generate exception handling code for our target language. Also note that the stl classes vector and string are used. SWIG generates code to wrap tcl parameters and converts them into the expected parameters for our SAL interface.

As you can see, we have achieved our goal to have very little code in this module. In fact, even the exception handling code is part of the generated code by SWIG. We do nothing here. One may then question what happens with the exceptions, where are they handled. This question is answered in the next section.

2.3 Target Scripting Language Library Module

This module is target language specific and at the moment we only have the tcl specific module. This module first loads the SWIG Interface Library dll and makes those functions exportable from a namespace we call stc::

The parameters can be checked here when necessary and the exception handling code can be added on the functions we dim necessary. At the moment, since the next task is to handle errors, we have not yet determined which functions need to provide a mechanism to automatically “discard” or at least log and ignore any possible exception, because we do not know yet which ones will need this functionality. But some cases are provided below to illustrate possible issues.

Most functions should be left without handling exceptions. This will call the user’s attention that something wrong has happened. For example, using the stc::create function, if it fails, this is really bad. An exception will be raised, the user’s script is terminated with enough information to correct the error, for example the cause of the error and possibly suggestion on how to fix it. This seems to also be fine for other functions, such as stc::get, stc::config, stc::delete and most others. In those cases we’ll want possibly the exact same behavior.

Now, suppose that we call a function to start a test. In this case the above behavior would be unacceptable. We do not want to terminate an otherwise ok test, which say it’s been running for several hours, only to find out that a trivial error caused an exception to be raised. In this case, an acceptable option would be to log the error and continue, and somehow possibly notify the user.

The difficulty of this is to know when to choose one method or the other. On one hand, the safest thing would seem to be the later option. However, it has the potential of becoming too diluted and would be hard for the user to determine when some serious error happened. Using the first option exclusively is not an option since it will cause problems for users.

On first analysis, these are the commands that are dimmed safe to raise exceptions:
init, shutdown, connect, disconnect, create, delete, config, get, reserve, release, subscribe, unsubscribe

These are functions that should never raise an exception:
log, logwarn, logerror and logdebug

perform should raise exceptions, but this may depend on which function it is performing. For now, it seems that it should raise exceptions for any function.

Other commands that are not yet part of the interface, but will become part are:
apply, start and stop

The commands start and stop, should probably raise exceptions. And even the apply command should raise them as well. So one may wonder what commands should not, that is hard to say. One

thing to note is the fact that the BLL will notify SAL of some events, and some of these events may be error conditions. So it is important that these events are handled so that the users can be notified appropriately. However, if this events are serious errors, the question is, should those events cause exceptions to be raised? In my opinion, a resilient program should try to continue in spite of error conditions. However, does that mean we continue for any error condition? That's also hard to say. What we do know is that at the very least this conditions or events should be made known to the user.

2.4 Limitations

These modules are to work as a single instance; only one script will be allowed to run at any one time. The scripts will also be limited to connecting to a local copy of the BLL. That is, both Bll.Base.dll and Sal.dll must reside in the same machine.

In future versions of this software, it is envisioned that a script could run and connect to a BLL that resides on another machine. It may also be possible then, to be able to run multiple scripts simultaneously.

3.0 Design Requirements

The language dependent library may have to contain target language specific code. However, we should try to minimize this code and keep the interface as light as possible. The goal being zero lines of code; only the necessary lines to wrap the functions and rename them so the target language can use them in the form stc::function_name.

Tcl supports namespaces, but other languages may not. So this wrapper library will have to contain whatever is necessary to make it work for the target language.

Currently, it seems there will be a functions stc::run which when executed, it must return. But we would have to check it continuously until it finishes. For this, we may need a “helper” function in the target language. Or we could do something similar to stc::perform, which in SAL, it executes and returns immediately, and we keep checking until it completes, and then we can return to the user.

3.1 Tools

SWIG from www.swig.org was used to generate the wrapper code for this project.

This tool was added to the mainline in the common/tools directory and the requirement is that there be an executable swig binary file that is generated from the source for each and every target platform. Currently, we have a binary for the windows build. We need to compile a binary for linux and one for solaris, and possibly for freebsd.

3.2 Dependencies

The project creates unidirectional dependencies as follows:

- This project depends on SAL and during linking it requires SAL.lib. Also it must have access to the SAL’s include directory for access to SalInterface.h
- The SWIG Interface Library also must have access to the include location for the target language and possibly link to other object files.

4.0 Unit Test Strategy

Because this project virtually has no code, there are no “unit” tests to conduct. The only code available is the generated code and our team in several discussions dimmed unnecessary to unit test code that we did not produce. So, in a sense we trust the tool SWIG to work sufficiently well. In fact, in the SWIG forums, the tool appears to work quite well, and there didn’t appear to be any serious issues.

The one thing I could observe about the code produced by the tool it’s its reliance on legacy “C” code, which in itself is not a problem, and it in fact from the purely performance oriented perspective, may be faster. Also the tool creates class objects for several operations. The one thing I’d like to be able to know is whether these objects are disposed of properly. Otherwise it has the potential of having memory leaks.

As we have mentioned above, there are no unit tests for this project and the only way to test these modules is through using the target language and write scripts that exercise all exported functions from SAL. We have those, including negative tests that cause errors that raise exceptions and then we have observed the expected behavior.

Another concern would be performance. We should test this mechanism and see some metrics, and try to pinpoint any possible bottle necks.

5.0 Acronyms

BLL	Business Logic Layer
ERD	Engineering Requirements Document
ERQ	Engineering Requirement
IL	Instruments Layer
PEP	Product Evolution Process
PL	Presentation Layer
TBA	To be added
SAL	Spirent Automation Layer
SWIG	Simple Wrapper Interface Generator

,